Presentation Script

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Slide 1 – Title 0:00–0:35

Good morning respected chairpersons, distinguished guests, and fellow researchers.

I am Debopam Sinha from the Department of Electronics and Communication Engineering, PES University, Bengaluru.

Today, I will present mywork titled *"LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection."*

The central question behind this work is simple: can we use a modern large language model to detect and localize Hardware Trojans at RTL level, while keeping the computational requirement practical for an academic research environment?

Slide 2 – Presentation Outline 0:35–0:50

I will first briefly introduce the motivation and background.

Then I will explain the proposed workflow, dataset construction, training configuration, evaluation measures, and results.

Finally, I will discuss the limitations and future scope of this work.

Slide 3 – Introduction 0:50–1:35

Modern integrated circuits increasingly rely on third-party IP cores, hardware reuse, and complex design workflows.

This creates a serious security concern: a Hardware Trojan can be inserted into a design and remain inactive during normal verification. It may activate only under a rare trigger condition.

RTL-level analysis is valuable because it enables security assessment before synthesis and physical implementation.

However, conventional detection methods can become difficult to scale for complex or obfuscated RTL designs.

Recent LLM-based approaches are promising because Verilog is a structured language. But full model adaptation can require expensive GPUs and large computational resources.

mywork addresses this resource challenge using LoRA-based parameter-efficient fine-tuning.

Slide 4 – Background 1:35–2:10

A Hardware Trojan usually contains two important components.

The first is the **trigger**. This is a rare condition that activates the Trojan.

The second is the **payload**. This performs the malicious or unintended behavior after activation.

For example, a trigger may activate only when a specific counter value and input condition occur together. The payload may then alter a signal or modify the intended computation.

LLMs are relevant here because transformer models can learn relationships within serialized RTL code. This makes them useful candidates for RTL understanding, Trojan detection, and localization.

Slide 5 – Proposed Work 2:10–3:05

This slide shows the complete proposed workflow.

We begin with clean Verilog RTL samples. First, the RTL undergoes preprocessing, including formatting normalization, serialization, and lightweight structural obfuscation.

Next, controlled trigger and payload logic are inserted into selected RTL modules. During this stage, we record the inserted trigger and payload regions as ground-truth metadata.

Then, prompts are prepared for the model. Importantly, temporary Trojan markers, explicit annotations, and Trojan-specific comments are removed before final model use. This reduces the possibility of shortcut learning from obvious labels.

The base model is Qwen2.5-1.5B-Instruct. Instead of updating the entire model, we use LoRA adapters and keep most pretrained parameters frozen.

Finally, the adapted model performs two tasks: Trojan classification and localization of the trigger and payload regions.

Slide 6 – Dataset Description 3:05–3:45

mydataset was constructed from clean Verilog RTL samples obtained from the RTL-Coder repository.

We selected 300 clean RTL samples as the base set.

Using controlled Trojan insertion, we generated 300 Trojan-inserted RTL samples. Therefore, the final dataset is balanced, containing 600 samples in total: 300 clean and 300 Trojan samples.

We used a randomized stratified split:

- 420 samples for training,
- 90 samples for validation,
- 90 samples for testing.

Each split preserves an equal number of clean and Trojan samples.

Slide 7 – Trojan Injection Example 3:45–4:20

This slide illustrates a clean Verilog counter module and an example of controlled Trojan insertion.

The trigger condition activates when `en` is high and the counter reaches decimal value 10.

The payload signal becomes active after the trigger condition is satisfied.

This controlled design allows us to preserve valid RTL behavior while creating a known Trojan region for evaluation.

The purpose is not to claim that this represents every real-world Trojan. It provides a reproducible controlled environment for evaluating detection and localization.

Slide 8 – Metadata and Prompt Sanitization 4:20–4:55

Here, the inserted trigger and payload logic are shown together with the metadata recorded during generation.

The `trigger_nodes` and `payload_nodes` metadata identify the inserted RTL regions and provide ground truth for localization evaluation.

A key point is that markers such as `HT_START`, `HT_END`, and Trojan-specific comments are generation-time annotations only.

They are removed during final prompt preparation. Therefore, the model does not receive explicit Trojan markers during inference.

Slide 9 – Training Configuration 4:55–5:35

For model adaptation, we used Qwen2.5-1.5B-Instruct with LoRA-based PEFT.

The LoRA rank is 8. We used FP16 precision, a learning rate of 2×10^{-4}, batch size 1, gradient accumulation of 8, and trained for 3 epochs.

Only 2.17 million parameters were trainable. This is approximately 0.141 percent of the full model parameters.

The implementation ran on Google Colab using an NVIDIA Tesla T4 GPU, with a training runtime of approximately six minutes.

This configuration demonstrates that the workflow can be reproduced without dedicated high-end computing infrastructure.

Slide 10 – Performance Measures 5:35–6:15

We evaluate binary Trojan detection using fmystandard metrics: accuracy, precision, recall, and F1-score.

Accuracy measures overall classification correctness.

Precision indicates how reliable a Trojan prediction is.

Recall measures how many actual Trojan samples are detected.

The F1-score balances precision and recall.

For localization, we use Trigger Localization Coverage, or TLC, and Payload Localization Coverage, or PLC.

TLC measures the fraction of ground-truth trigger nodes covered by predicted trigger nodes. PLC uses the same principle for payload nodes.

Slide 11 – Results and Analysis 6:15–7:25

On the held-out test set of 90 samples, the framework achieved 98.89 percent accuracy and 98.88 percent F1-score.

Precision was 100 percent, which means that no clean sample was incorrectly predicted as Trojan in this test set.

Recall was 97.78 percent. One Trojan sample was predicted as clean.

The confusion matrix shows that all 45 clean samples were correctly classified. Among the 45 Trojan samples, 44 were correctly detected and one was missed.

For localization, both TLC and PLC reached 98.89 percent under this controlled evaluation.

From the resource perspective, the method used LoRA-based PEFT, 2.17 million trainable parameters, a Tesla T4 GPU, and approximately six minutes of runtime.

These results should be interpreted within the controlled synthetic RTL environment used in this study.

Slide 12 – Conclusion and Future Scope 7:25–8:25

To conclude, we developed a resource-efficient LoRA-based framework for RTL Hardware Trojan detection and localization.

The work combines controlled Trojan generation, prompt sanitization, parameter-efficient LLM adaptation, and localization-aware evaluation.

On the controlled held-out test set, the framework achieved 98.89 percent accuracy, 98.88 percent F1-score, and 98.89 percent TLC and PLC.

The main contribution is not only the detection performance. It is also the ability to perform this experiment with a lightweight adaptation strategy on a Tesla T4 GPU.

At the same time, we acknowledge the current limitations.

The dataset is synthetic, the analysis is limited to RTL level, and the randomized split does not explicitly enforce template-level separation.

Future work will evaluate larger and more heterogeneous benchmarks, real-world Trojan datasets, stronger generalization protocols, and gate-level or post-synthesis analysis.

Slide 13 – References 8:25–8:40

These are the key references that informed this work, including PEARL, TrojanWhisper, LoRA, RTL-Coder, and foundational Hardware Trojan literature.

Slide 14 – Thank You 8:40–8:50

Thank you for ymyattention.

I will be happy to answer any questions.

Conference Defence Q&A Handbook

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Debopam Sinha

How to answer in the conference: Answer directly in two or three sentences. Do not over-explain unless the professor asks a follow-up. For a limitation, acknowledge it confidently and state the next experiment that would address it.

1. Basic Concepts

Q: What is a Hardware Trojan?
Answer: A Hardware Trojan is an intentional malicious modification of a hardware design. It can remain dormant during normal operation and activate only under a particular condition to alter functionality, leak information, degrade performance, or create another security risk.

Q: What is RTL?
Answer: RTL stands for Register Transfer Level. It is a hardware-design abstraction that describes how data moves between registers and how combinational logic processes that data, usually using Verilog or VHDL.

Q: Why is RTL-level Trojan detection important?
Answer: RTL enables security analysis early in the design flow, before synthesis and physical implementation. Detecting suspicious logic at this stage can reduce the cost and difficulty of correcting a design later.

Q: What is the difference between RTL, gate level, and physical design?
Answer: RTL describes intended functionality using constructs such as registers, conditions, and assignments. Gate level represents the synthesized netlist of logic gates. Physical design represents placement and routing on silicon. Our work evaluates only RTL-level analysis.

Q: What are a trigger and a payload in a Hardware Trojan?
Answer: The trigger is the condition that activates the Trojan, often a rare combination of inputs or internal states. The payload is the malicious behavior that occurs after activation.

Q: Can you give a simple trigger-payload example from your work?
Answer: In the illustrative counter example, the trigger becomes active when `en = 1` and the counter reaches decimal value 10. The payload then affects the counter behavior through an additional signal.

Q: Why can Hardware Trojans escape normal verification?
Answer: Verification usually tests expected operating conditions and practical test scenarios. A Trojan can use a rare trigger condition that is not exercised during normal simulation or testing.

Q: What is the difference between a bug and a Hardware Trojan?
Answer: A bug is generally an unintended design error. A Hardware Trojan is intentionally inserted malicious logic. Their observable behavior can sometimes look similar, but their origin and threat model differ.

Q: Why use Verilog in this work?
Answer: Verilog is widely used for RTL design and is a structured hardware description language. Its code-like form also makes it suitable for transformer-based language-model processing.

Q: What does Trojan detection mean in your paper?
Answer: Trojan detection is binary classification of an RTL sample as clean or Trojan-inserted. Our model predicts whether a given serialized RTL prompt contains suspicious Trojan logic.

Q: What does Trojan localization mean in your paper?
Answer: Localization means identifying RTL regions associated with the Trojan trigger and payload. We compare predicted regions against ground-truth metadata recorded during controlled Trojan generation.

2. Motivation and Research Contribution

Q: What problem does your research solve?
Answer: The work studies whether an LLM can perform RTL Trojan detection and localization with a resource-efficient adaptation method. The aim is reproducible academic experimentation without requiring large-scale GPU infrastructure.

Q: What is the main contribution of this work?
Answer: The main contribution is an end-to-end controlled RTL Trojan analysis workflow that combines synthetic Trojan generation, prompt sanitization, LoRA-based adaptation, detection, and localization-aware evaluation in a low-resource setting.

Q: What is novel in your work?
Answer: The novelty is primarily in the engineering integration of a controlled RTL Trojan dataset, label-sanitized prompt preparation, LoRA adaptation of Qwen2.5-1.5B-Instruct, and trigger-payload localization evaluation under constrained hardware. I do not claim a fundamentally new transformer architecture.

Q: Why did you choose an LLM instead of a traditional machine-learning model?
Answer: RTL is structured sequential code with relationships across declarations, conditions, and assignments. A code-capable transformer can learn contextual patterns in serialized RTL, while conventional handcrafted-feature methods may require manual feature design.

Q: Does an LLM understand hardware in the same way as an engineer?
Answer: No. The LLM learns statistical and structural patterns from the representation and training data. It can support security analysis, but its outputs require careful evaluation and should not replace formal hardware-security verification.

Q: Why not use only formal verification?
Answer: Formal verification is powerful and provides stronger guarantees for specified properties, but it can be computationally expensive and requires suitable properties or threat assumptions. Our approach is complementary: it is a learned screening and localization workflow, not a replacement for formal verification.

Q: Why not use side-channel Trojan detection?
Answer: Side-channel methods often require fabricated hardware or measured signals such as power, timing, or electromagnetic behavior. RTL analysis operates earlier in the design process and does not require a physical chip.

Q: Why is resource efficiency important?
Answer: Many academic researchers do not have access to dedicated high-memory GPUs or multi-GPU servers. Parameter-efficient adaptation lowers the number of trainable parameters and makes experimental work more reproducible in such environments.

3. Dataset and Trojan Generation

Q: Where did the clean RTL data come from?
Answer: The clean Verilog RTL samples were collected from the RTL-Coder repository. They were used as the base designs for controlled Trojan generation.

Q: How many samples are in the dataset?
Answer: The final balanced dataset contains 600 RTL samples: 300 clean samples and 300 Trojan-inserted samples.

Q: How did you create Trojan samples?
Answer: We inserted controlled trigger and payload logic into clean Verilog RTL modules while aiming to preserve valid RTL functionality. The insertion process also recorded trigger and payload regions as metadata.

Q: Why use synthetic Trojans instead of real-world Trojans?

Answer: Synthetic generation provides known ground truth for both detection and localization, which supports controlled and reproducible evaluation. However, it is also a major limitation because synthetic Trojans do not cover the full diversity of real-world attack strategies.

Q: Does your dataset contain real industrial Hardware Trojans?
Answer: No. The current work uses controlled synthetic RTL Trojan samples. Testing on heterogeneous real-world benchmarks is an important future direction.

Q: How is the dataset split?
Answer: We used a randomized stratified split. Training contains 420 samples, validation contains 90 samples, and testing contains 90 samples. Each split has equal clean and Trojan class counts.

Q: Why did you use stratified splitting?
Answer: Stratification preserves the clean-to-Trojan class balance in each split. This prevents class imbalance from artificially affecting the classification metrics.

Q: Is the test set balanced?
Answer: Yes. The test set contains 45 clean RTL samples and 45 Trojan-inserted RTL samples.

Q: Could similar RTL templates appear in both training and test sets?
Answer: Yes, that possibility cannot be ruled out with a randomized stratified split. The paper explicitly acknowledges that template-level separation was not enforced, so the results should be interpreted as performance in a controlled setting rather than proof of broad structural generalization.

Q: How would you improve the split in future work?
Answer: I would use template-level, design-family-level, or project-level separation. This would ensure that structurally similar designs do not appear across training and test partitions.

Q: How did you ensure that the Trojan insertion did not break RTL functionality?
Answer: The framework uses controlled, syntactically valid insertion patterns intended to preserve valid RTL behavior. However, the paper does not claim exhaustive formal equivalence checking for every sample, so a stronger future workflow should add compilation, simulation, and formal validation after every insertion.

Q: Could your synthetic generation process create artifacts that make classification easier?
Answer: Yes, that is a valid concern. We reduced obvious artifacts through normalization, lightweight structural obfuscation, and removal of explicit Trojan markers. But synthetic-generation bias can still remain, which is why broader benchmarks and stronger split protocols are necessary.

4. Prompt Preparation and Leakage

Q: What is prompt preparation in your work?
Answer: Prompt preparation converts serialized RTL into an instruction-oriented representation suitable for the LLM. It includes formatting normalization, whitespace cleanup, and removal of explicit annotations that could leak labels.

Q: What are `HT_START` and `HT_END` markers?
Answer: They are temporary markers used during the controlled generation process to identify inserted Trojan regions. They are not retained in the final prompts used by the model.

Q: Why remove Trojan-specific comments and markers?
Answer: If explicit markers remained, the model could learn to classify the label from those markers instead of reasoning from RTL structure. Removing them reduces this obvious shortcut-learning risk.

Q: Can you call the evaluation completely leakage-free?
Answer: I would use the more precise phrase *label-sanitized evaluation*. Explicit labels, markers, and Trojan comments were removed from final prompts, but randomized splitting does not completely eliminate possible structural similarity between training and test samples.

Q: Were ground-truth localization labels given to the model during inference?

Answer: No. The trigger and payload metadata were retained separately for evaluation. During inference, the model received prompt information without disclosure of the ground-truth localization annotations.

Q: How could leakage still occur even after marker removal?
Answer: Leakage can occur through repeated insertion templates, characteristic coding styles, similar RTL designs across splits, or other systematic artifacts. Marker removal addresses only explicit annotation leakage.

Q: Why use obfuscation-aware preprocessing?
Answer: The purpose is to reduce dependence on superficial naming patterns or fixed textual artifacts. The goal is to encourage the model to rely more on RTL structural relationships.

Q: Did you prove that the model learned true hardware semantics?
Answer: No. The reported results show strong performance within the controlled evaluation setup, but they do not prove full semantic hardware understanding. Additional out-of-distribution and template-separated experiments would be needed.

5. LLM, LoRA, and Training

Q: What is Qwen2.5-1.5B-Instruct?
Answer: It is an instruction-tuned large language model with approximately 1.5 billion parameters. We selected it because it is moderately sized and supports instruction-oriented learning workflows.

Q: Why did you choose Qwen2.5-1.5B-Instruct?
Answer: It provides a practical balance between model capacity and resource requirements. A much larger model would increase memory and training cost, while a much smaller model might have less capacity for RTL pattern learning.

Q: What is fine-tuning?
Answer: Fine-tuning adapts a pretrained model to a specific domain or task using task-specific data. In this work, the target task is RTL Trojan detection and localization.

Q: What is LoRA?
Answer: LoRA stands for Low-Rank Adaptation. Instead of updating all pretrained model weights, it adds small trainable low-rank adapter matrices while keeping most backbone parameters frozen.

Q: Why does LoRA reduce computational cost?
Answer: LoRA updates only a small subset of parameters. This reduces optimizer-state memory, gradient storage, and training computation compared with full fine-tuning.

Q: What does PEFT mean?
Answer: PEFT means Parameter-Efficient Fine-Tuning. LoRA is one widely used PEFT method.

Q: What is the difference between LoRA and full fine-tuning?
Answer: Full fine-tuning updates most or all model parameters. LoRA freezes most pretrained parameters and trains only lightweight adapter parameters.

Q: What is the difference between LoRA and QLoRA?
Answer: LoRA adds low-rank trainable adapters to a model. QLoRA combines this approach with low-bit quantization of the base model. The final workflow reported in this paper used FP16 LoRA adaptation for runtime stability and reproducibility.

Q: What was the LoRA rank in your work?
Answer: The LoRA rank was 8. Rank controls the capacity of the low-rank adapter update. A larger rank can increase adaptation capacity but also increases trainable parameters and resource use.

Q: Which exact transformer layers did you adapt with LoRA?
Answer: The paper reports the LoRA configuration, rank, and trainable parameter count, but it does not provide a complete layer-by-layer target-module specification. I would not claim unreported implementation details. A public reproducibility package should document the exact target modules.

Q: What does FP16 mean?
Answer: FP16 means 16-bit floating-point precision. It reduces memory use and can improve training speed compared with 32-bit floating-point computation, subject to numerical-stability considerations.

Q: What learning rate did you use?
Answer: The learning rate was 2×10^{-4}, also written as `2e-4`.

Q: What were the key training settings?
Answer: The reported configuration used LoRA rank 8, FP16 precision, batch size 1, gradient accumulation 8, three training epochs, and a learning rate of 2×10^{-4}.

Q: Why was batch size set to 1?
Answer: A batch size of 1 helps keep GPU memory use low when adapting an LLM on a Tesla T4. Gradient accumulation of 8 helps approximate a larger effective batch while preserving the memory constraint.

Q: What is gradient accumulation?
Answer: Instead of updating the model after every individual sample, gradients are accumulated over multiple steps before an optimizer update. Here, accumulation of 8 supports training stability while retaining a physical batch size of 1.

Q: How many trainable parameters were used?
Answer: The LoRA configuration used 2.17 million trainable parameters, reported as 0.141 percent of the total model parameters.

Q: Why is the trainable percentage so small?
Answer: Most Qwen backbone weights remain frozen. Only the LoRA adapter parameters are optimized for the RTL Trojan task.

Q: What hardware did you use?
Answer: The workflow ran in Google Colab on an NVIDIA Tesla T4 GPU.

Q: Is a Tesla T4 a consumer GPU?
Answer: Strictly speaking, the Tesla T4 is a data-center GPU. The point of the paper is that it is accessible through common academic cloud environments such as Google Colab, rather than requiring a dedicated high-end multi-GPU server.

Q: What does the reported six-minute runtime include?
Answer: The paper reports an approximate training runtime of six minutes for the stated controlled setup. Runtime can vary with Colab allocation, data preparation, environment initialization, and implementation details, so it should be treated as an approximate reproducibility indicator rather than a universal benchmark.

6. Metrics and Results

Q: What are the test-set results?
Answer: On the 90-sample controlled test set, the framework achieved 98.89% accuracy, 100.00% precision, 97.78% recall, and 98.88% F1-score.

Q: What was the confusion matrix?
Answer: There were 45 clean samples correctly predicted as clean, zero clean samples incorrectly predicted as Trojan, one Trojan sample predicted as clean, and 44 Trojan samples correctly predicted as Trojan.

Q: Why is the accuracy 98.89%?
Answer: There were 89 correct predictions out of 90 test samples. Therefore, accuracy is 89/90, which equals approximately 98.89%.

Q: Why is the precision 100%?
Answer: Precision for the Trojan class is $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. There were 44 true Trojan predictions and zero false-positive Trojan predictions, so precision is $44/(44+0) = 100\%$.

Q: Why is recall 97.78%?
Answer: Recall for the Trojan class is $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. There were 44 detected Trojan samples and one missed Trojan sample, so recall is $44/(44+1) = 97.78\%$.

Q: Why is F1-score lower than precision?
Answer: F1-score is the harmonic mean of precision and recall. Although precision is 100%, recall is lower because one Trojan sample was missed.

Q: Which error is more serious for Hardware Trojan detection: false positive or false negative?
Answer: Both matter, but they have different costs. A false negative may allow a Trojan to remain undetected, while a false positive can create unnecessary verification effort. In security-sensitive applications, false negatives are often especially concerning.

Q: What are TLC and PLC?
Answer: TLC is Trigger Localization Coverage, and PLC is Payload Localization Coverage. They measure how much of the ground-truth trigger or payload region is covered by the predicted RTL region.

Q: What is the TLC formula?
Answer: The paper uses:

$$\mathrm{TLC} = \frac{|T_p \cap T_g|}{|T_g|}$$

where T_p is the predicted trigger-node set and T_g is the ground-truth trigger-node set.

Q: What is the PLC formula?
Answer: The paper uses:

$$\mathrm{PLC} = \frac{|P_p \cap P_g|}{|P_g|}$$

where P_p is the predicted payload-node set and P_g is the ground-truth payload-node set.

Q: Why are TLC and PLC both 98.89%?
Answer: They are the reported trigger and payload coverage results under this controlled evaluation. Since these are node-level coverage measures, the percentage should not automatically be interpreted as one missed sample out of 90.

Q: Does high TLC or PLC prove exact localization?
Answer: No. TLC and PLC are coverage-oriented measures. They show how much of the ground-truth region the prediction covers, but they do not by themselves strongly penalize predicting extra irrelevant nodes.

Q: What additional localization metrics would strengthen the work?
Answer: Localization precision, intersection-over-union, exact-match rate, and region-level F1-score would provide a more complete assessment. They would quantify both coverage and unnecessary predicted regions.

Q: Why did you not report confidence intervals?
Answer: The current paper reports point estimates on a 90-sample test set. Confidence intervals, repeated runs with different seeds, and statistical comparisons would strengthen future evaluation, especially because the test set is modest in size.

Q: Can you claim 98.89% accuracy in all real RTL designs?
Answer: No. The correct claim is that the framework achieved 98.89% accuracy on the controlled randomized test set used in this study. It should not be generalized to all real-world RTL or industrial hardware without broader validation.

7. Comparison, Validity, and Limitations

Q: How does your work compare with PEARL?
Answer: PEARL provides important context for LLM-based Hardware Trojan analysis. However, it uses a

different model, adaptation strategy, evaluation setting, and hardware environment. Therefore, our paper treats it as a contextual reference rather than a directly comparable benchmark.

Q: Why did you not directly compare accuracy with PEARL?
Answer: A direct comparison would not be scientifically fair without using the same dataset, split, threat model, prompts, task definition, and evaluation protocol. Different experimental settings can make numerical comparisons misleading.

Q: Does your work outperform PEARL?
Answer: I would not make that claim. The focus of our work is resource-efficient LoRA-based adaptation under a controlled setup, not direct performance competition against PEARL.

Q: Did you compare against a classical machine-learning baseline?
Answer: The accepted paper focuses on establishing the controlled LoRA-based prototype and does not report a matched classical baseline. Adding baselines such as rule-based detection, graph-based models, or non-adapted LLM prompting would strengthen future work.

Q: What is the biggest limitation of your work?
Answer: The biggest limitation is external validity. The dataset uses controlled synthetic Trojans, and the randomized split does not enforce template-level separation. Therefore, strong results within this setup do not yet establish broad real-world generalization.

Q: Is the paper production-ready for industrial chip security?
Answer: No. It is an engineering-oriented research prototype. Industrial deployment would require evaluation on real-world benchmarks, gate-level and physical-design validation, robust threat modeling, formal checks, and integration with established verification workflows.

Q: Can an attacker evade your detector?
Answer: Potentially, yes. An attacker may use a structurally different trigger, stealthier payload, adversarial RTL formatting, or logic that differs from the synthetic templates. Robustness against adaptive adversaries requires dedicated security evaluation.

Q: Could the model hallucinate a suspicious RTL region?
Answer: Yes. LLM outputs can be imperfect or overconfident. Localization should therefore guide human review or downstream verification rather than act as an unquestioned security verdict.

Q: Does your method prove that an RTL design is Trojan-free?
Answer: No. A classifier cannot provide a formal proof of Trojan absence. The model can prioritize suspicious designs or regions for further investigation, but formal verification and expert analysis remain necessary.

Q: Can this approach detect a Trojan introduced after synthesis?
Answer: Not in its current form. The present work is limited to RTL. Gate-level netlists and post-synthesis modifications require separate representations, datasets, and evaluation methods.

Q: Can this approach detect analog or physical Trojans?
Answer: No. The current approach analyzes serialized RTL and does not model analog effects, layout-level manipulations, doping-level Trojans, or supply-chain physical attacks.

Q: Why should reviewers trust the result despite these limitations?
Answer: The results are useful because the dataset construction, split, configuration, and limitations are stated clearly. The paper presents evidence for a controlled low-resource prototype, while avoiding the unsupported claim that it solves all Hardware Trojan detection problems.

8. Tough Professor Questions

Q: Your accuracy is very high. Why should I believe it is not caused by dataset artifacts?
Answer: That concern is valid. We removed explicit markers and Trojan-specific comments, and used preprocessing to reduce superficial shortcuts. However, synthetic-template artifacts and structural simi-

larity may still remain, so the results should be interpreted as controlled-setting performance. Template-separated and external-benchmark evaluation are the necessary next steps.

Q: If the dataset is synthetic, are you detecting Trojans or merely detecting your own insertion pattern?
Answer: The controlled setup evaluates whether the model can detect and localize the inserted RTL patterns after explicit label artifacts are removed. It does not prove performance against every attacker-designed Trojan. That distinction is exactly why the paper frames the work as a reproducible prototype and proposes real-world benchmark evaluation as future work.

Q: Why should a professor accept this as research rather than a simple fine-tuning exercise?
Answer: The value lies in the complete security-analysis workflow: controlled Trojan generation with localization ground truth, sanitization to reduce obvious leakage, parameter-efficient adaptation, and joint detection-localization evaluation under limited hardware. The research contribution is an evaluated systems and methodology contribution, not a claim of inventing LoRA or transformers.

Q: What would happen if you removed LoRA and used the base model without adaptation?
Answer: That would be an important ablation study. The accepted paper does not report that result, so I would not invent a number. A future ablation should compare zero-shot prompting, few-shot prompting, LoRA adaptation, and possibly full fine-tuning under the same split.

Q: What would happen if you changed the LoRA rank from 8?
Answer: The trade-off would likely be between adaptation capacity and resource cost. Lower ranks may reduce trainable parameters but limit adaptation, while higher ranks may improve task fit at increased memory and training cost. This should be tested systematically through an ablation study.

Q: Why only three epochs?
Answer: Three epochs were part of the reported low-resource configuration. More epochs may improve fitting or may increase overfitting, especially with a small controlled dataset. Validation-based model selection and a learning-curve study would make this decision stronger.

Q: Why is the test set only 90 samples?
Answer: The 600-sample balanced dataset was split into 420 training, 90 validation, and 90 testing samples. The test set is sufficient for the reported controlled experiment, but it remains modest. Larger external test sets and repeated trials are necessary for stronger statistical confidence.

Q: Why did you not use k-fold cross-validation?
Answer: The study used a fixed randomized stratified split to maintain a simple and reproducible experimental protocol. Cross-validation, especially grouped cross-validation by design template, would provide a stronger estimate of generalization in future work.

Q: Can TLC and PLC be high even when the model predicts many extra nodes?
Answer: Yes. Because TLC and PLC use ground-truth coverage in the denominator, they mainly measure how much relevant ground truth is recovered. Extra predicted nodes are not fully penalized, so localization precision or IoU should be added in future work.

Q: What is your exact threat model?
Answer: The current threat model assumes controlled RTL-level insertion of trigger and payload logic into clean Verilog modules. It does not claim coverage of all adversarial strategies, post-synthesis attacks, or physical supply-chain attacks.

Q: What should be the next immediate experiment after this paper?
Answer: The most important next experiment is a template-separated evaluation on larger and more diverse RTL benchmarks, combined with matched baselines. This would test whether performance remains strong when structural similarity across training and test data is reduced.

Q: If you had more GPU resources, what would you change?
Answer: I would first increase experimental rigor rather than simply use a larger model. I would run ablations, multiple random seeds, grouped splits, external benchmarks, localization precision metrics, and comparisons with full fine-tuning or other PEFT configurations.

9. Final Defence Statements to Memorize

If asked about the 98.89% result:
"The framework achieved 98.89% accuracy on our controlled 90-sample randomized test set. I present it as a strong result within that experimental setting, not as a universal claim for all real-world RTL Trojans."

If asked about novelty:
"The contribution is an end-to-end, resource-efficient RTL Trojan analysis workflow that combines controlled generation, label-sanitized prompts, LoRA adaptation, and localization-aware evaluation."

If asked about the main limitation:
"The main limitation is that the dataset is synthetic and the randomized split does not explicitly enforce template-level separation. The next step is grouped evaluation on broader and more realistic benchmarks."

If asked whether this replaces formal verification:
"No. This work is best viewed as an intelligent screening and localization aid. Formal verification and expert review remain essential for security-critical hardware assurance."

If you do not know an answer:
"That specific implementation detail is not reported in the accepted paper, so I do not want to make an unsupported claim. Based on the current methodology, the correct next step would be to evaluate it through a controlled ablation or validation experiment."

Advanced Defence Q&A

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Rule for hard questions: Never become defensive. State the evidence, acknowledge the limitation, and explain the scientifically correct next experiment.

1. Dataset Leakage and Experimental Validity

Q: You call the evaluation leakage-safe. How can you claim that when synthetic generation may leave recognizable patterns?
Defence answer: The precise claim is that we removed *explicit label leakage*: temporary `HT_START`, `HT_END`, Trojan-specific comments, and direct annotations were removed from final prompts. However, synthetic insertion can still create structural artifacts. Therefore, the results demonstrate label-sanitized performance in a controlled setting, not complete immunity to all dataset leakage.

Q: Could the model simply detect your Trojan template instead of detecting a general Hardware Trojan?
Defence answer: That is a valid threat to external validity. The experiment proves that the model can detect controlled Trojan-inserted RTL patterns after explicit labels are removed. It does not prove general detection of every real attacker-designed Trojan. Template-separated and external-benchmark evaluation are necessary next steps.

Q: Why did you not enforce template-level separation between training and test data?
Defence answer: The current study used a randomized stratified split to preserve class balance and provide a controlled first evaluation. The paper explicitly identifies the absence of template-level separation as a limitation. A stronger future protocol would group related RTL designs and insertion templates into separate partitions.

Q: If similar RTL modules exist in training and testing, is your reported accuracy inflated?
Defence answer: It may be optimistic because structural similarity across splits cannot be ruled out. That is why I present 98.89% as controlled randomized-test performance rather than a universal generalization result.

Q: Why should we trust a 90-sample test set?
Defence answer: The test set gives a clear controlled result: 89 correct predictions out of 90. However, 90 samples is still modest for broad statistical claims. Larger held-out benchmarks, confidence intervals, repeated random seeds, and grouped cross-validation would strengthen confidence.

Q: Why did you not report confidence intervals?
Defence answer: The paper reports point estimates. Confidence intervals would be valuable because they show uncertainty caused by finite test-set size. This should be included in a more extensive follow-up study.

Q: Did you check for duplicate RTL files across data splits?
Defence answer: The paper does not report a duplicate-detection or near-duplicate analysis, so I would not claim that it was performed. In future work, I would use code similarity analysis and design-family grouping before splitting.

Q: Can preprocessing itself create a new leakage signal?
Defence answer: Yes. Any deterministic preprocessing pipeline can introduce regularities. The role of normalization and obfuscation is to reduce superficial shortcuts, but only robust ablations and external testing can show whether those shortcuts have truly been reduced.

2. Model Choice and LoRA

Q: Why use an LLM for this task instead of a graph neural network or a traditional classifier?
Defence answer: RTL contains code-like sequential structure, dependencies, conditions, and assignments that code-capable transformers can model from serialized text. Graph neural networks are also a strong alternative because circuits naturally form graphs. A fair comparison would require the same dataset and evaluation protocol, and this is a valuable future baseline.

Q: Why Qwen2.5-1.5B-Instruct specifically?
Defence answer: It offers a reasonable balance of language-model capacity and practical resource use. The study targets low-resource academic experimentation, so a 1.5-billion-parameter instruction model is more feasible than much larger models.

Q: Why did you not use a hardware-specialized pretrained model?
Defence answer: The selected model provides a practical instruction-tuned base for the controlled experiment. A hardware-specialized model could be an important comparison in future work, particularly to test whether domain-specific pretraining improves generalization.

Q: What is the mathematical intuition behind LoRA?
Defence answer: Instead of learning a full weight update ΔW, LoRA approximates it using low-rank matrices:

$$\Delta W = BA,$$

where A and B have much smaller rank than the original weight matrix. The adapted weight becomes $W' = W + BA$, while the pretrained W remains frozen.

Q: Why does LoRA reduce memory use?
Defence answer: Full fine-tuning stores gradients and optimizer states for nearly all model parameters. LoRA only optimizes small adapter matrices, so the training-state memory requirement is much lower.

Q: Does LoRA guarantee the same performance as full fine-tuning?
Defence answer: No. LoRA is a trade-off between adaptation capacity and resource efficiency. It can perform very well, but full fine-tuning may outperform it in some tasks or datasets at a substantially higher computational cost.

Q: Why did you choose LoRA rank 8?
Defence answer: Rank 8 was the reported configuration for this low-resource prototype. It gives a compact adapter capacity. A complete study should include rank ablations, for example ranks 4, 8, 16, and 32, to quantify the accuracy-resource trade-off.

Q: Which exact layers were targeted by LoRA adapters?
Defence answer: The accepted paper reports LoRA rank, the PEFT strategy, and trainable parameter count, but it does not document the target transformer modules in detail. I would not claim a specific target-module configuration without implementation evidence.

Q: Why use FP16 rather than INT8 or 4-bit quantization?
Defence answer: The paper notes exploratory QLoRA-style low-bit adaptation, but the final reported workflow used FP16 LoRA adaptation for runtime stability and reproducibility. Low-bit quantization can reduce memory further, but it can also introduce implementation and stability considerations.

Q: Why train for only three epochs?
Defence answer: Three epochs were part of the reported resource-constrained configuration. More epochs could improve task fit or could overfit a small synthetic dataset. A validation-based learning-curve study would provide a stronger justification.

3. Detection Metrics and Localization

Q: Your accuracy is high. Could it be misleading?
Defence answer: Yes. Accuracy can be misleading in an imbalanced dataset, although our test set is balanced with 45 clean and 45 Trojan samples. That is why we also report precision, recall, F1-score, the confusion matrix, TLC, and PLC.

Q: Why is precision 100% while recall is lower?
Defence answer: The model produced no false-positive Trojan predictions, so Trojan-class precision is 100%. It missed one actual Trojan sample, so recall is $44/45 = 97.78\%$.

Q: Which is more dangerous: a false positive or a false negative?
Defence answer: A false negative can allow a Trojan to remain undetected, which is serious in a security context. A false positive increases verification cost and may waste engineer time. The preferred trade-off depends on the security-critical application.

Q: Why did you not optimize for recall if missed Trojans are dangerous?
Defence answer: The reported results are from the selected classification configuration rather than an explicitly recall-optimized threshold study. A deployment-oriented system should assess threshold tuning, cost-sensitive learning, and the acceptable false-negative rate for the target threat model.

Q: What exactly does TLC measure?
Defence answer: TLC measures trigger-node coverage:

$$\mathrm{TLC} = \frac{|T_p \cap T_g|}{|T_g|},$$

where T_p is the predicted trigger-node set and T_g is the ground-truth trigger-node set. It measures how much of the actual trigger region is recovered.

Q: What exactly does PLC measure?
Defence answer: PLC measures payload-node coverage:

$$\mathrm{PLC} = \frac{|P_p \cap P_g|}{|P_g|},$$

where P_p is the predicted payload-node set and P_g is the ground-truth payload-node set.

Q: Can TLC and PLC be high if the model predicts many irrelevant nodes?
Defence answer: Yes. TLC and PLC are coverage-oriented metrics. They reward recovering ground-truth nodes but do not strongly penalize extra predicted nodes. Future work should add localization precision, region-level F1-score, exact-match rate, and intersection-over-union.

Q: Why is localization ground truth reliable in this study?
Defence answer: Ground truth is reliable with respect to the controlled insertion procedure because the system records the inserted trigger and payload regions as metadata. This reliability applies to the synthetic experiment, not necessarily to naturally occurring or unknown real-world Trojans.

Q: Does high localization coverage prove that the model understands circuit semantics?
Defence answer: No. High coverage demonstrates agreement with recorded synthetic ground truth in the evaluation setup. Semantic understanding requires stronger evidence, such as testing on novel design families, novel Trojan types, and independently labeled real-world benchmarks.

Q: How does the model output a localization prediction?
Defence answer: The paper evaluates predicted suspicious RTL regions against recorded trigger and payload metadata. The paper does not provide a complete formal specification of the output parser or region-decoding mechanism, so I would not claim details beyond the reported methodology.

4. Threat Model and Security Questions

Q: What is your exact threat model?
Defence answer: The work assumes controlled RTL-level insertion of trigger and payload logic into clean Verilog modules. It evaluates whether an adapted model can detect and localize these modifications after explicit annotations are removed.

Q: Can your method detect a Trojan inserted by a sophisticated adversary who knows your training method?
Defence answer: Not yet established. An adaptive adversary could use structurally different logic, rare coding styles, delayed activation, or adversarial transformations. Robustness against adaptive attackers requires dedicated adversarial evaluation.

Q: Can an attacker evade the model by renaming signals or reformatting code?
Defence answer: Renaming and formatting transformations are relevant concerns. The preprocessing pipeline attempts to reduce dependence on superficial textual artifacts, but systematic robustness testing against semantics-preserving transformations is needed.

Q: Can this method detect dormant Trojans that never activate in simulation?
Defence answer: Potentially, RTL analysis can inspect suspicious structural patterns without requiring runtime activation. However, detection performance for arbitrary dormant Trojans is not established by this controlled dataset alone.

Q: Can this approach detect analog Trojans, doping Trojans, or layout-level attacks?
Defence answer: No. The current work is limited to RTL text representations. Analog, doping, gate-level, and physical-layout Trojans require different data, representations, and security-analysis techniques.

Q: Can your system formally prove that a chip is Trojan-free?
Defence answer: No. This is a learned detection and localization aid, not a formal proof system. Formal verification, equivalence checking, security review, and physical testing remain necessary for high-assurance hardware security.

Q: Could the LLM hallucinate a Trojan location?
Defence answer: Yes. LLM-generated predictions can contain incorrect or overconfident outputs. Therefore, the system should prioritize regions for expert review or downstream formal checks rather than make an autonomous final security decision.

5. Comparison and Reproducibility

Q: Why did you not directly compare with PEARL?
Defence answer: PEARL uses a different model, adaptation strategy, hardware environment, dataset, and evaluation setting. Comparing final accuracy values without matching those conditions would be scientifically weak. We use PEARL as contextual prior work, not as a directly matched benchmark.

Q: Does your result outperform PEARL?
Defence answer: I do not claim that. Our work emphasizes low-resource LoRA-based experimentation, whereas PEARL has a different research focus and experimental setup.

Q: Why did you not compare against a zero-shot LLM baseline?
Defence answer: That would be a useful ablation. The accepted paper focuses on the controlled LoRA-based framework and does not report a zero-shot baseline. A stronger future study should compare zero-shot, few-shot, LoRA, full fine-tuning, and classical hardware-security baselines.

Q: Why did you not compare against traditional ML or GNN models?
Defence answer: A matched comparison would make the study stronger. It requires identical data splits, preprocessing, and evaluation metrics to be meaningful. This is a clear future-work direction.

Q: Can another researcher reproduce your six-minute runtime?

Defence answer: They may obtain a similar order of runtime under comparable conditions, but exact runtime can vary due to GPU allocation, Colab environment, software versions, initialization, and data loading. The reported value is approximate, not a guaranteed constant.

Q: Is Google Colab suitable for reproducible research?
Defence answer: It is useful for accessible experimentation, but reproducibility should also document package versions, random seeds, data-processing code, model configuration, and exact training commands. Cloud hardware availability can vary.

Q: What would a complete reproducibility package contain?
Defence answer: It should contain the clean-data source references, controlled Trojan-generation code, preprocessing and sanitization code, split files, random seeds, LoRA configuration, training scripts, inference and localization parser, metric code, and environment specifications.

6. Strong Closing Answers

Q: What is the one sentence contribution of your paper?
Defence answer: We present a controlled, resource-efficient RTL Trojan detection and localization workflow using LoRA adaptation of Qwen2.5-1.5B-Instruct, evaluated on a balanced synthetic RTL dataset with explicit limitations stated.

Q: What is the one sentence limitation of your paper?
Defence answer: The strong results come from a controlled synthetic RTL setting with a randomized split, so they do not yet prove generalization to structurally novel, real-world, gate-level, or physical-design Trojans.

Q: What is the most important future experiment?
Defence answer: A template-separated evaluation on larger and heterogeneous real-world RTL Trojan benchmarks, with matched classical, graph-based, zero-shot, and PEFT baselines.

Q: Why is this work still useful despite its limitations?
Defence answer: It establishes a reproducible starting point for studying LLM-based RTL security analysis under limited hardware resources. It also exposes the exact validation gaps that future work must address before deployment claims are justified.

Master Technical Q&A for Conference Defence

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Debopam Sinha

Defence rule: State exactly what the paper demonstrates. Do not claim broader generalization than the experiment supports. If a limitation is raised, acknowledge it clearly and state the next experiment needed.

1. Hardware Security Fundamentals

Q: What is a Hardware Trojan?
Answer: A Hardware Trojan is an intentional malicious modification of a hardware design. It can remain inactive during normal operation and activate under a rare internal state or input condition.

Q: Why are Hardware Trojans dangerous?
Answer: They can alter functionality, leak sensitive information, reduce reliability, create a backdoor, or cause denial of service. They are dangerous because they may remain dormant during normal verification.

Q: What is RTL?
Answer: RTL stands for Register Transfer Level. It describes digital hardware behavior in terms of registers, data flow, conditional logic, and clocked operations, commonly using Verilog or VHDL.

Q: Why perform Trojan detection at RTL level?
Answer: RTL-level analysis occurs early in the design flow, before synthesis and physical implementation. Early detection can reduce redesign cost and supports security review before fabrication.

Q: What is the difference between RTL, gate-level netlist, and layout?
Answer: RTL describes intended logic behavior. A gate-level netlist describes synthesized logic gates and connections. Layout describes the physical placement and routing of those gates on silicon. This work focuses only on RTL.

Q: What is a Trojan trigger?
Answer: A trigger is the condition that activates a Trojan. It may depend on a rare input sequence, a counter value, an internal state, or a combination of signals.

Q: What is a Trojan payload?
Answer: A payload is the malicious action that occurs after the trigger activates. It may modify data, alter control logic, leak a secret, or disrupt normal operation.

Q: What is the difference between a bug and a Trojan?
Answer: A bug is generally unintentional. A Trojan is intentionally inserted malicious logic. Both may create incorrect behavior, but their security implications and threat models are different.

Q: Why might a Trojan escape normal simulation?
Answer: A Trojan can use a rare trigger condition that standard test benches do not activate. Simulation also cannot exhaustively test all possible states in a complex design.

Q: Can RTL analysis detect every Hardware Trojan?
Answer: No. RTL analysis cannot cover all Trojans, especially analog, gate-level, doping-level, or physical-layout attacks. It is one layer of a broader hardware-security workflow.

Q: What are conventional methods for Hardware Trojan detection?
Answer: Common approaches include functional testing, formal verification, side-channel analysis, structural analysis, machine learning, and deep learning. Each method has different requirements and limitations.

Q: Why not rely only on formal verification?
Answer: Formal verification can provide strong guarantees for specified properties, but it may be computationally expensive and requires a suitable property specification. The proposed method can help screen suspicious RTL regions for further formal analysis.

2. Motivation and Research Contribution

Q: What problem does this research address?
Answer: This work studies RTL Hardware Trojan detection and localization using an LLM while keeping adaptation practical under limited computing resources.

Q: What is the main objective of the work?
Answer: The objective is to build a resource-efficient framework that classifies RTL as clean or Trojan-inserted and identifies suspicious trigger and payload regions.

Q: What is the central contribution?
Answer: The central contribution is an end-to-end controlled framework combining RTL preprocessing, synthetic Trojan generation, prompt sanitization, LoRA-based adaptation, detection, and localization-aware evaluation.

Q: What is novel in this work?
Answer: The novelty is the integrated engineering workflow for controlled RTL Trojan analysis using lightweight LoRA adaptation and localization metadata. The work does not claim a new transformer architecture or a new LoRA algorithm.

Q: Why use an LLM for RTL analysis?
Answer: RTL is structured code containing declarations, conditions, assignments, and dependencies. Transformer-based LLMs can learn contextual relationships from serialized code-like representations.

Q: Does an LLM truly understand hardware semantics?
Answer: Not in the same way as a hardware engineer or a formal tool. It learns statistical and structural patterns from data. Strong results indicate useful task performance in the tested setting, not complete semantic understanding.

Q: Why is resource efficiency important?
Answer: Full LLM fine-tuning can require large GPUs and substantial memory. Resource efficiency makes experiments more accessible and reproducible for academic researchers.

Q: How is your work different from a normal code-classification task?
Answer: The task involves hardware-security reasoning over RTL, controlled Trojan trigger-payload insertion, and localization of suspicious RTL regions. The security interpretation and localization ground truth make it different from ordinary code classification.

3. Dataset Engineering

Q: Where did the clean RTL samples come from?
Answer: The clean Verilog RTL samples were obtained from the RTL-Coder repository. These samples form the base set for controlled Trojan generation.

Q: How large is the final dataset?
Answer: The final dataset contains 600 RTL samples: 300 clean samples and 300 Trojan-inserted samples.

Q: Why is the dataset balanced?
Answer: A balanced dataset prevents a classifier from gaining high accuracy merely by predicting the majority class. Here, clean and Trojan classes contain equal numbers of samples.

Q: How were Trojan-inserted samples created?
Answer: Controlled trigger and payload logic were inserted into clean Verilog RTL modules. The insertion process also recorded the affected trigger and payload regions as metadata.

Q: Why use synthetic Trojan generation?
Answer: Synthetic generation provides known ground truth for both classification and localization. It also makes the experiment reproducible, although it limits real-world generalization.

Q: Does the dataset contain real industrial Trojans?

Answer: No. The current dataset uses controlled synthetic RTL Trojan insertions. Real-world and heterogeneous benchmark evaluation remains future work.

Q: How is the dataset split?
Answer: The split is randomized and stratified: 420 samples for training, 90 for validation, and 90 for testing. Each split has equal clean and Trojan class counts.

Q: How many clean and Trojan samples are in the test set?
Answer: The test set contains 45 clean samples and 45 Trojan-inserted samples.

Q: Why use a randomized stratified split?
Answer: Randomization supports a conventional held-out evaluation, while stratification preserves class balance across training, validation, and test sets.

Q: Does randomized splitting guarantee generalization?
Answer: No. Similar RTL structures or insertion patterns may still appear across splits. The paper explicitly states that template-level separation was not enforced.

Q: Could related RTL templates appear in both training and testing data?
Answer: Yes, that cannot be ruled out. This may make the reported performance optimistic relative to a template-separated or cross-project evaluation.

Q: How should future work improve the split?
Answer: Future work should use design-family-level, template-level, or project-level splits. This would better test performance on structurally novel RTL designs.

Q: How did you preserve valid RTL functionality after Trojan insertion?
Answer: The procedure used controlled and syntactically valid insertion patterns intended to preserve valid RTL behavior. However, the paper does not claim exhaustive functional equivalence checking, so future work should add compilation, simulation, and formal validation for every generated sample.

4. Trojan Injection and Prompt Sanitization

Q: What is the purpose of recording `trigger_nodes` and `payload_nodes`?
Answer: They provide ground-truth localization metadata. During evaluation, predicted suspicious regions are compared against these recorded trigger and payload regions.

Q: What are `HT_START` and `HT_END` markers?
Answer: They are temporary generation-time markers used to identify inserted Trojan regions. They are not part of the final model prompt.

Q: Why remove explicit Trojan markers before training and inference?
Answer: If markers or Trojan-specific comments remained, the model could classify samples using obvious textual labels instead of RTL structure. Removing them reduces explicit shortcut learning.

Q: What is prompt sanitization?
Answer: Prompt sanitization removes temporary markers, Trojan-specific comments, and explicit annotations while retaining the RTL structural content needed for analysis.

Q: Does removing markers eliminate all data leakage?
Answer: No. It removes explicit label leakage, but synthetic insertion templates, coding patterns, or similar RTL structures may still create indirect leakage.

Q: Can you call the evaluation completely leakage-free?
Answer: The safer term is *label-sanitized evaluation*. Explicit annotations were removed, but the randomized split does not fully exclude structural similarity across partitions.

Q: Are localization labels shown to the model during inference?
Answer: No. The ground-truth trigger and payload metadata are retained separately for evaluation. During inference, the model receives the RTL prompt without the ground-truth localization annotations.

Q: Are classification labels shown to the model during training?
Answer: The model requires task supervision during adaptation, but the key leakage-control point is that the input RTL prompt does not contain direct Trojan markers or embedded ground-truth annotations. At inference, labels are hidden.

Q: What is structural obfuscation in this context?
Answer: It refers to lightweight transformations intended to reduce reliance on superficial naming or formatting patterns while preserving RTL structural relationships.

Q: Can preprocessing itself introduce artifacts?
Answer: Yes. Any deterministic processing pipeline can create regularities. Robust ablations and external evaluation are needed to verify that preprocessing improves rather than biases generalization.

5. LLM and Transformer Concepts

Q: What is a large language model?
Answer: A large language model is a neural network trained to model token sequences. It learns statistical relationships between tokens and can be adapted for tasks such as classification, generation, code analysis, and question answering.

Q: What is a transformer?
Answer: A transformer is a neural-network architecture that uses attention mechanisms to model relationships between tokens in a sequence. It is the main architecture behind modern LLMs.

Q: What is self-attention?
Answer: Self-attention allows each token to assign importance to other tokens in the sequence. In RTL, this can help the model connect signals, conditions, assignments, and declarations that may be far apart in the text.

Q: Why is attention useful for RTL?
Answer: RTL contains long-range dependencies. For example, a signal may be declared in one place, assigned elsewhere, and used in a conditional block later. Attention can model these relationships.

Q: What is tokenization?
Answer: Tokenization converts text into smaller units called tokens that the model can process. Verilog symbols, keywords, identifiers, operators, and numbers are represented as token sequences.

Q: Can tokenization create challenges for RTL?
Answer: Yes. Long identifiers, code formatting, uncommon syntax, and long modules can increase token length or fragment meaningful RTL constructs. This is another reason why code-aware preprocessing matters.

Q: What is Qwen2.5-1.5B-Instruct?
Answer: It is an instruction-tuned LLM with approximately 1.5 billion parameters. It was selected because it provides practical model capacity with manageable resource requirements.

Q: Why choose a 1.5-billion-parameter model?
Answer: It provides a balance between capability and accessibility. Much larger models may be more costly to adapt, while much smaller models may have less capacity for complex RTL pattern learning.

Q: Why use an instruction-tuned model?
Answer: Instruction-tuned models are designed to follow task-oriented prompts. This aligns with the instruction-oriented RTL prompt format used in the workflow.

Q: Could a graph neural network be better for this task?
Answer: Possibly. Hardware designs naturally form graphs of signals and logic dependencies. A graph neural network is an important baseline, but fair comparison requires the same dataset, split, labels, and evaluation protocol.

6. LoRA, PEFT, and Training

Q: What is fine-tuning?
Answer: Fine-tuning adapts a pretrained model to a domain-specific task using task-specific data. In this work, the target task is RTL Trojan detection and localization.

Q: What is PEFT?
Answer: PEFT means Parameter-Efficient Fine-Tuning. It adapts a model while updating only a small fraction of parameters.

Q: What is LoRA?
Answer: LoRA stands for Low-Rank Adaptation. It adds small trainable low-rank matrices to selected model weights while keeping most pretrained model parameters frozen.

Q: What is the mathematical idea behind LoRA?
Answer: Instead of fully learning a large update ΔW, LoRA approximates it using:

$$\Delta W = BA,$$

where A and B are low-rank trainable matrices. The adapted weight becomes:

$$W' = W + BA.$$

The original pretrained weight W remains frozen.

Q: Why does LoRA save memory?
Answer: The optimizer, gradients, and updates are stored mainly for small adapter matrices rather than for the full backbone. This substantially reduces the trainable state compared with full fine-tuning.

Q: What is the difference between LoRA and full fine-tuning?
Answer: Full fine-tuning updates most or all model parameters. LoRA updates only small adapter parameters while preserving the base model weights.

Q: Does LoRA always perform as well as full fine-tuning?
Answer: No. LoRA trades adaptation capacity for efficiency. It can be highly effective, but full fine-tuning may perform better for some tasks at much higher computational cost.

Q: What was the LoRA rank in your work?
Answer: The reported LoRA rank was 8. It controls the capacity of the low-rank adapter update.

Q: Why choose rank 8?
Answer: Rank 8 was used as the selected low-resource configuration. A more extensive study should compare multiple ranks to quantify the performance-resource trade-off.

Q: What is QLoRA?
Answer: QLoRA combines LoRA adapters with low-bit quantization of the base model. It can reduce memory use further, but the final reported workflow used FP16 LoRA adaptation for stability and reproducibility.

Q: What is FP16?
Answer: FP16 is 16-bit floating-point precision. It reduces memory use and can improve training speed compared with FP32, subject to numerical-stability considerations.

Q: What was the learning rate?
Answer: The learning rate was 2×10^{-4}, also written as `2e-4`.

Q: What was the batch size?
Answer: The physical batch size was 1. This keeps GPU memory usage low.

Q: What is gradient accumulation?
Answer: Gradient accumulation adds gradients over several steps before updating the parameters. The reported gradient accumulation value was 8.

Q: Why use batch size 1 with gradient accumulation 8?
Answer: This configuration allows training within a limited-memory GPU environment while approximating an effective larger batch for parameter updates.

Q: How many epochs were used?
Answer: The training configuration used 3 epochs.

Q: How many parameters were trainable?
Answer: The LoRA adapters contained 2.17 million trainable parameters, reported as 0.141% of the total model parameters.

Q: Why is only 0.141% trainable?
Answer: Most Qwen backbone parameters remain frozen. Only lightweight LoRA adapter parameters are optimized for the task.

Q: Which exact model layers did LoRA target?
Answer: The accepted paper reports the LoRA rank and trainable parameter count but does not provide a complete target-layer specification. I would not claim details that are not documented in the paper.

Q: What hardware was used?
Answer: The implementation used Google Colab with an NVIDIA Tesla T4 GPU.

Q: Is Tesla T4 a consumer GPU?
Answer: Tesla T4 is a data-center GPU. The practical point is that it is commonly accessible in academic cloud environments such as Google Colab, without requiring dedicated multi-GPU infrastructure.

Q: What does the approximately six-minute runtime mean?
Answer: It is the reported approximate training runtime for the controlled setup. Exact time can vary with cloud allocation, software versions, initialization, data loading, and hardware availability.

7. Detection and Localization Metrics

Q: What binary classification metrics are used?
Answer: The work reports accuracy, precision, recall, and F1-score.

Q: What is accuracy?
Answer: Accuracy is the fraction of all correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Q: What is precision?
Answer: Precision measures the reliability of positive Trojan predictions:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Q: What is recall?
Answer: Recall measures the fraction of actual Trojan samples that are detected:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Q: What is F1-score?
Answer: F1-score is the harmonic mean of precision and recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Q: What does a false positive mean in your task?
Answer: A false positive means that a clean RTL sample is incorrectly classified as Trojan-inserted.

Q: What does a false negative mean in your task?
Answer: A false negative means that a Trojan-inserted RTL sample is incorrectly classified as clean. In a hardware-security context, this can be particularly serious.

Q: What is TLC?
Answer: TLC means Trigger Localization Coverage. It measures how much of the ground-truth trigger region is covered by the predicted trigger region:

$$\mathrm{TLC} = \frac{|T_p \cap T_g|}{|T_g|}.$$

Q: What is PLC?
Answer: PLC means Payload Localization Coverage. It measures how much of the ground-truth payload region is covered by the predicted payload region:

$$\mathrm{PLC} = \frac{|P_p \cap P_g|}{|P_g|}.$$

Q: What do T_p and T_g represent?
Answer: T_p is the predicted trigger-node set, while T_g is the ground-truth trigger-node set recorded during controlled Trojan insertion.

Q: What do P_p and P_g represent?
Answer: P_p is the predicted payload-node set, while P_g is the recorded ground-truth payload-node set.

Q: Do TLC and PLC measure localization precision?
Answer: No. They are coverage-oriented metrics. They measure recovery of ground-truth nodes, but they do not fully penalize predicting extra irrelevant nodes.

Q: What additional localization metrics would strengthen the work?
Answer: Localization precision, exact-match rate, region-level F1-score, and intersection-over-union would provide a more complete evaluation.

8. Experimental Results

Q: What was the test-set accuracy?
Answer: The model achieved 98.89% accuracy on the 90-sample controlled test set.

Q: What was the test-set precision?
Answer: Precision was 100.00% for Trojan predictions.

Q: What was the test-set recall?
Answer: Recall was 97.78%.

Q: What was the F1-score?
Answer: The F1-score was 98.88%.

Q: What was the confusion matrix?
Answer: The test set contained 45 clean and 45 Trojan samples. The model correctly classified 45 clean samples and 44 Trojan samples. It produced zero false positives and one false negative.

Q: Why is accuracy 98.89%?
Answer: There were 89 correct predictions out of 90 test samples:

$$\frac{89}{90} \times 100 = 98.89\%.$$

Q: Why is precision 100%?
Answer: There were 44 true Trojan predictions and zero false-positive Trojan predictions:

$$\frac{44}{44+0} = 100\%.$$

Q: Why is recall 97.78%?
Answer: There were 44 detected Trojan samples and one missed Trojan sample:

$$\frac{44}{44+1} = 97.78\%.$$

Q: What were the localization results?
Answer: The reported Trigger Localization Coverage was 98.89%, and the reported Payload Localization Coverage was also 98.89%.

Q: Can you claim that the model is 98.89% accurate for all Hardware Trojans?
Answer: No. The correct statement is that the model achieved 98.89% accuracy on the controlled randomized test set used in this study.

Q: Why report both detection and localization?
Answer: Binary detection identifies whether a sample appears suspicious. Localization adds practical value by identifying the trigger and payload regions that may require engineering review.

9. Comparison, Limitations, and Threat Model

Q: How does this work compare with PEARL?
Answer: PEARL is important related work in LLM-based Hardware Trojan analysis. However, it uses a different model, adaptation approach, evaluation setting, and hardware environment, so it is treated as contextual prior work rather than a direct benchmark.

Q: Does your work outperform PEARL?
Answer: I do not claim that. The experiments are not directly matched, and comparing raw performance numbers across different datasets and protocols would be scientifically misleading.

Q: Why did you not compare with zero-shot prompting?
Answer: A zero-shot baseline would be a useful ablation. The accepted paper focuses on the controlled LoRA-based workflow and does not report that baseline.

Q: Why did you not compare with classical machine learning or graph neural networks?
Answer: Such comparisons would strengthen the evaluation. They should use identical splits, preprocessing, and metrics to make the comparison meaningful.

Q: What is the main limitation of the dataset?
Answer: The data are controlled synthetic Trojan samples. Synthetic patterns may not capture the diversity and stealth of real-world Hardware Trojans.

Q: What is the main limitation of the split?
Answer: The randomized stratified split does not enforce template-level separation. Therefore, structural similarity between training and test data cannot be ruled out.

Q: What is the main limitation of the localization metrics?
Answer: TLC and PLC measure coverage but do not fully measure false localization. Additional precision-oriented localization metrics are needed.

Q: What is the main limitation of the hardware scope?
Answer: The work analyzes RTL only. It does not evaluate gate-level netlists, post-synthesis representations, physical layouts, analog Trojans, or manufacturing-level attacks.

Q: Can an adaptive attacker evade this method?
Answer: Potentially, yes. An attacker may use different Trojan structures, uncommon RTL patterns, adversarial transformations, or post-synthesis modifications. Robustness against adaptive attackers needs dedicated testing.

Q: Can this work prove a design is Trojan-free?
Answer: No. This is a learned screening and localization tool, not a formal proof of hardware security. Formal verification and expert review remain necessary.

Q: Can the LLM hallucinate a suspicious location?
Answer: Yes. LLM predictions can be incorrect or overconfident. Localization should guide review and downstream verification rather than serve as an autonomous security verdict.

Q: Is this production-ready for industry?
Answer: No. It is a controlled engineering prototype. Production use would require real-world benchmarks, stronger baselines, grouped splits, formal validation, robustness tests, and integration with established verification flows.

10. Tough Professor Questions and Best Defence

Q: Are you detecting Trojans or merely detecting your own generated pattern?
Answer: The current experiment demonstrates detection and localization of controlled Trojan-inserted RTL after removal of explicit markers. It does not establish performance against every attacker-designed Trojan. That is why the paper presents the work as a reproducible prototype and identifies broader benchmark evaluation as future work.

Q: Your high result could be due to template leakage. Why should we accept it?
Answer: That concern is valid. We reduce explicit leakage through sanitization, but the randomized split cannot fully eliminate structural similarity. The result is still useful as controlled evidence for the proposed workflow, while template-separated evaluation is needed before making stronger generalization claims.

Q: Why should this be considered research rather than just fine-tuning a model?
Answer: The contribution is not merely applying LoRA. It is the design and evaluation of a complete controlled RTL security-analysis workflow that includes Trojan generation, localization metadata, prompt sanitization, lightweight adaptation, and explicit limitations.

Q: What would be the most important ablation study?
Answer: The most important ablation would compare zero-shot prompting, LoRA adaptation, full fine-tuning, and non-LLM baselines under the same template-separated dataset split.

Q: What would be the most important next experiment?
Answer: The most important next experiment is evaluation on larger, heterogeneous, template-separated, and ideally real-world RTL Trojan benchmarks.

Q: What would happen if you increased LoRA rank?
Answer: A higher rank would increase adapter capacity and trainable parameters. It might improve performance or might overfit the small synthetic dataset. The accuracy-resource trade-off should be measured empirically.

Q: What would happen if you used a larger LLM?
Answer: A larger model may have more capacity, but it would increase memory, training time, and cost. The key question is whether the performance improvement justifies the additional resource requirement.

Q: What would happen if you used full fine-tuning?
Answer: Full fine-tuning may provide more adaptation capacity but requires substantially more memory and computation. A direct comparison would quantify whether LoRA gives a favorable performance-efficiency trade-off.

Q: How would you validate functional correctness of generated Trojan RTL?
Answer: I would add automated compilation, simulation with regression test benches, linting, and formal

equivalence or property checking. This would strengthen confidence that each generated sample is valid and that the intended design behavior is controlled.

Q: If you do not know an implementation detail during the presentation, what should you say?
Answer: I would say: "That exact detail is not reported in the accepted paper, so I do not want to make an unsupported claim. The appropriate way to establish it would be through a documented ablation or reproducibility experiment."

11. Final Statements to Memorize

One-sentence contribution:
"We present a controlled, LoRA-based, resource-efficient framework for RTL Hardware Trojan detection and trigger-payload localization using Qwen2.5-1.5B-Instruct."

One-sentence result:
"On our controlled balanced 90-sample test set, the framework achieved 98.89% accuracy, 98.88% F1-score, and 98.89% trigger and payload localization coverage."

One-sentence limitation:
"The dataset is synthetic and the randomized split does not enforce template-level separation, so the results should not yet be generalized to all real-world Hardware Trojans."

One-sentence future work:
"The next step is template-separated evaluation on larger and more diverse real-world RTL benchmarks, supported by matched baselines and stronger localization metrics."

One-sentence safety claim:
"This method is an intelligent screening and localization aid; it complements formal verification and expert review rather than replacing them."